



Programs: Electrical & Computer Engineering

Course Number	ELE632
Course Title	Signal and Systems II
Semester/Year	Winter/2026
Instructor	Kevin Lowell Pham kevin.lowell.pham@torontomu.ca

Lab Report No.	1
-----------------------	----------

Report Title	Time-Domain Analysis of Discrete-Time Systems (Part 1)
---------------------	---

Section No.	ELE632 - 052 (Section 5)
Group No.	N/A
Submission Date	Jan 28, 2026
Due Date	Feb 1, 2026

Name	Student ID	Signature*
Syed Maisam Abbas	xxxx03255	

(Note: remove the first 4 digits from your student ID)

*By signing above you attest that you have contributed to this submission and confirm that all work you have contributed to this submission is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: <https://www.torontomu.ca/content/dam/senate/policies/pol60.pdf>

Table of Contents

Introduction.....	3
PART A: Signal Transformation.....	3
Question 1.....	3
Question 2.....	4
Question 3.....	6
PART B: Recursive Solution of difference equation.....	7
Question 1.....	7
Question 2.....	8
Question 3.....	9
PART C: Design a filter (N-point maximum filter).....	10
Question 1.....	10
Question 2.....	10
Question 3.....	11
PART D: Energy and power of a discrete signal.....	12
Question 1.....	12
Question 2.....	12
Appendix.....	13

INTRODUCTION

This lab assignment investigates the basic concepts of discrete-time signal processing to analyze mathematical models and digital implementations. In order to determine the physical attributes of discrete signals, the experiment first characterizes them using a variety of transformations and visualizations. Additionally, the energy and power of the signals are quantitatively assessed. Furthermore, the scope extends to system-level analysis, where the zero-input, zero-state, and total system responses are computed iteratively by deriving equations using the given functions.

The applications of how discrete systems respond to both input/output circumstances and algorithmic models are a crucial basis for research in digital controls and communications.

PART A: Signal Transformation

Question 1:

I. $\delta[n - 3]$

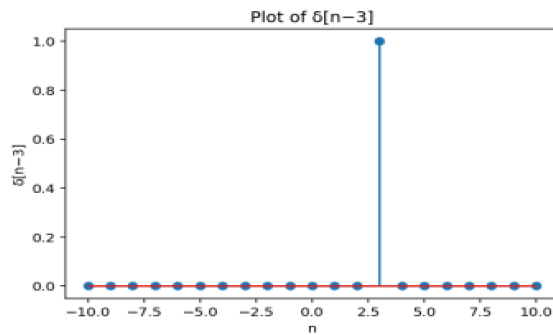


Figure 1: $\delta[n - 3]$

II. $u[n + 1]$

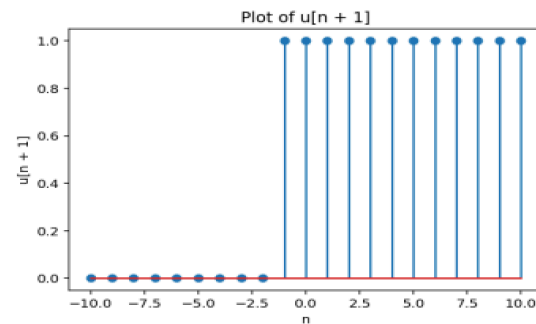


Figure 2: $u[n + 1]$

III. $x[n] = \cos\left(\frac{\pi n}{5}\right) u[n]$

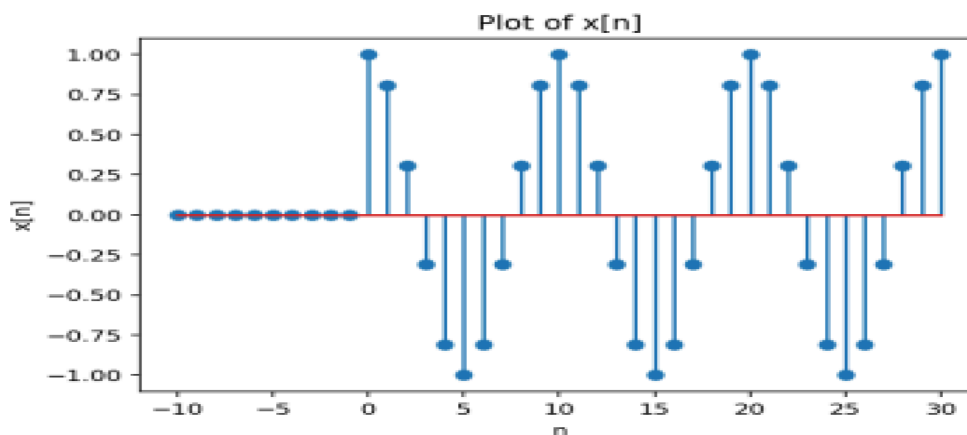


Figure 3: $x[n] = \cos\left(\frac{\pi n}{5}\right) u[n]$

IV. $x_1[n] = x[n - 3]$

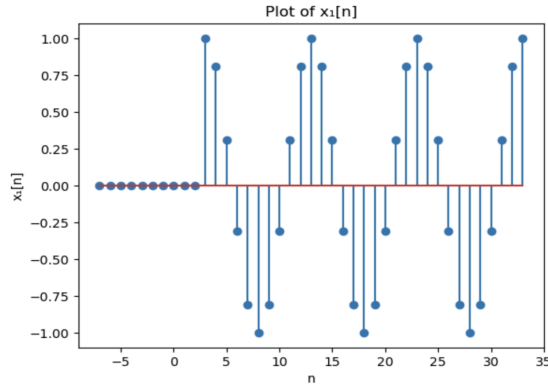


Figure 4: $x_1[n] = x[n - 3]$

V. $x_2[n] = x[-n]$

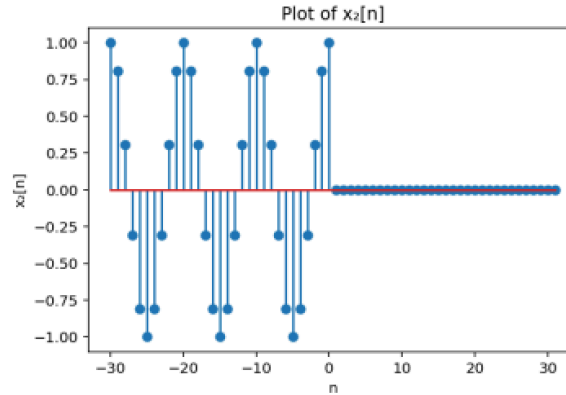


Figure 5: $x_2[n] = x[-n]$

The function $x[n] = \cos(\frac{\pi n}{5})u[n]$ represented in Figure 3 experiences a transformation performed by $x_1[n]$ & $x_2[n]$. The function $x_1[n] = x[n - 3]$ displays a time shift of 3 units to the left (a delay, towards the positive x-axis) as represented in Figure 4. This time delay is noticed by comparing the scale of the x-axis as it shifts to the left by 3 units. The function $x_2[n] = x[-n]$ represented in Figure 5 displays a time inversion, which is a reversal or reflection along the y-axis, as comparing the plots in Figures 3 and 5.

Question 2:

I. $y[n] = 5e^{-\frac{n}{8}} (u[n] - u[n - 10])$

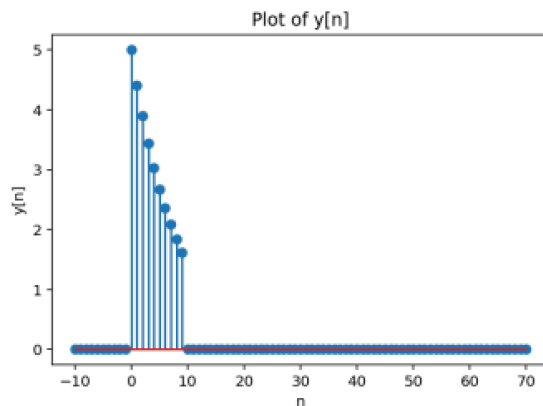


Figure 6: $y[n] = 5e^{-\frac{n}{8}} (u[n] - u[n - 10])$

II. $y_1[n] = y[3n]$

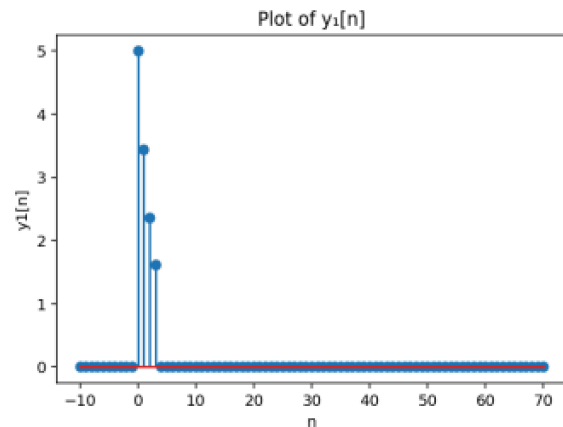


Figure 7: $y_1[n] = y[3n]$

III. $y_2[n] = y[\frac{n}{3}]$

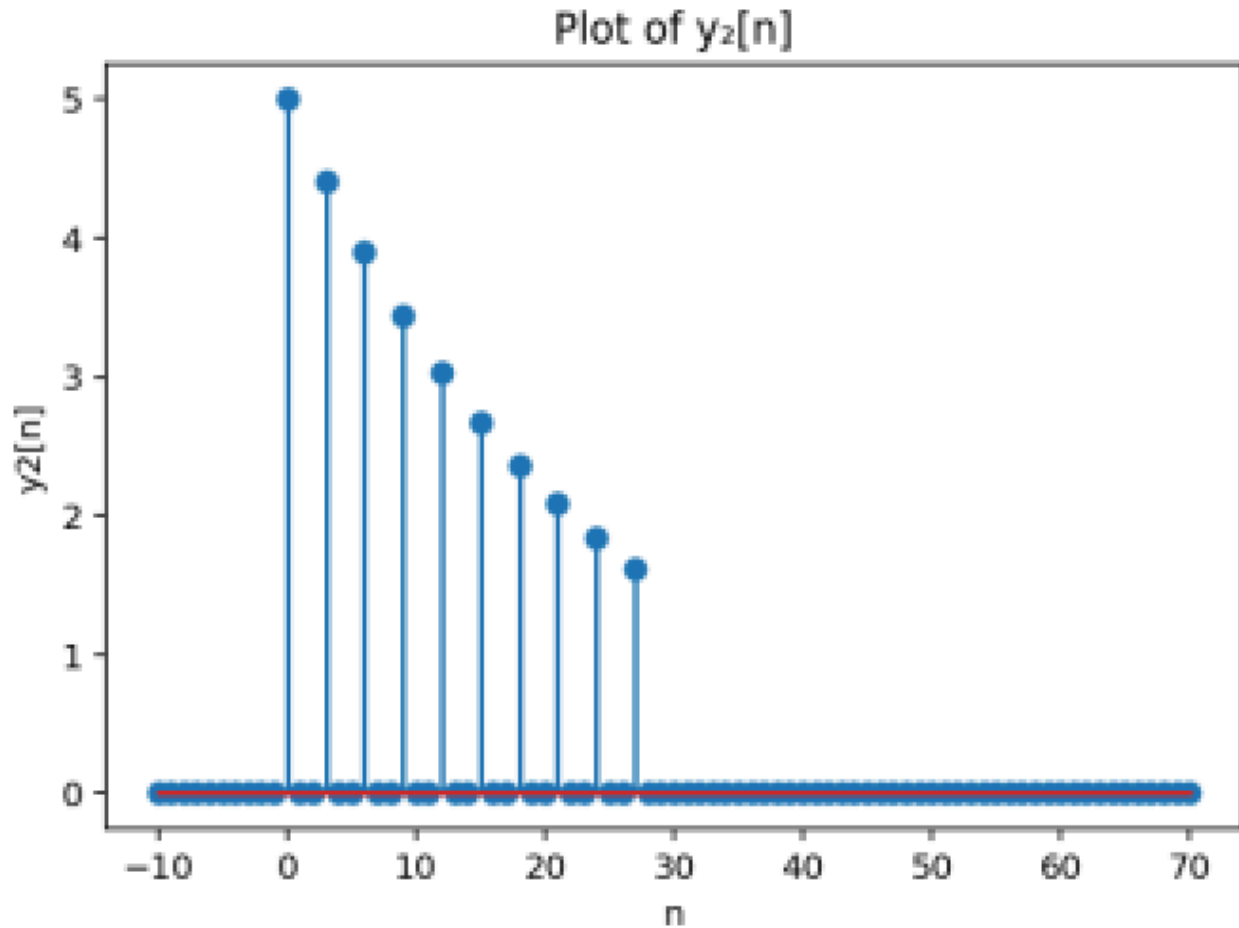


Figure 8: $y_2[n] = y[\frac{n}{3}]$

The function $y[n] = 5e^{-\frac{n}{8}}(u[n] - u[n - 10])$ represented on page 8 experiences a transformation of time scaling performed by $y_1[n]$ & $y_2[n]$. The function $y_1[n] = y[3n]$ displays a compression of 3 units as represented in Figure 7. The function $y_2[n] = y[\frac{n}{3}]$ represented in Figure 8 displays an expansion by a factor of $\frac{1}{3}$ as comparing the plots in Figures 6 and 8. This is also noticed by comparing the scale of the plots (the x-axis) as the scale factors a range from 3 to 30, respectively, while originating from 10 for the $y[n]$ graph.

Question 3:

I.

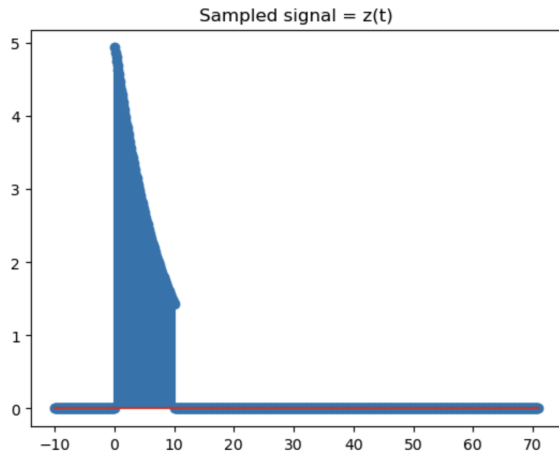


Figure 9: Sampled Signal

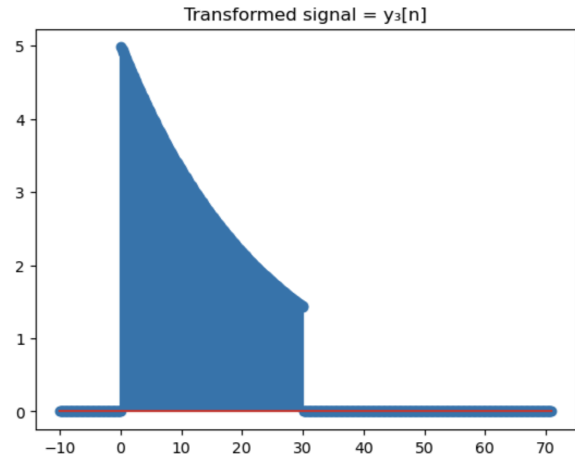


Figure 10: Transformed Signal

II.

$y_2[n]$ represented in Figure 8 and $y_3[n]$ (the Transformed Signal in Figure 10) are not the same due to sampling. Similarly, $z(t)$ (the Sampled Signal) and $y[n]$ represented on page 8 differ due to sampling multiple instances of n over the same period. The rate of sampling differs for

$y[n] = z(t) = 5e^{-\frac{n}{8}} (u[n] - u[n - 10])$ as $y[n]$ generates a total of 10 samples over a 10-unit cycle, having a default increment factor of 1. As for $z(t)$, the given range is selected with an increment of a factor of 0.1, therefore $z(t)$ generates a total of 100 samples for the same 10-unit cycle. Similarly, $y_2[n]$ being given the default increment of 1, generates a total of 30 samples (including the zeros) for the expanded transformation of $y[n]$ (with a factor of $\frac{1}{3}$).

Although $y_3[n]$ also shares the similarity of this expansion of a factor of $\frac{1}{3}$, the rate of sampling is given by $\frac{n}{0.1}$, where n is the scale. Thus, $\frac{30}{0.1} = 300 \text{ samples}$ for $y_3[n]$ as compared to $\frac{30}{1} = 30 \text{ samples}$ for $y_2[n]$.

PART B: Recursive Solution of difference equation

Question 1:

Student ID = 501103255 $\rightarrow 5 + 0 + 1 + 1 + 0 + 3 + 2 + 5 + 5 = 22$. Thus, $I = 5$ & $D = 1$.

The bank pays $(I + 1)\%$ interest on the account balance, and the account balance at the beginning of the year is given by $(D + 1) \times \$1000$. Therefore, $(5 + 1)\% = 6\% = 0.06$ as the interest and $(1 + 1) \times \$1000 = (2)(\$1000) = \$2000$ as the initial balance.

The equation relating the output balance (\$) = $y[n]$ to the inputs, deposit (\$) = $x[n]$, is given below. Where the interest rate is multiplied by the balance for the previous month, in each month of the year, as a loop, including the addition of the deposits:

<pre>for k in range(1, len(n)): y[k] = (1 + r) * y[k-1] + x[k]</pre>	Balance = (Interest)(prev. month) + Deposits + Initial Balance $y[n] = 1.06y[n - 1] + x[n] + 2000$
--	---

Equation 1: Equation relation the balance ($y[n]$) to the deposits ($x[n]$)

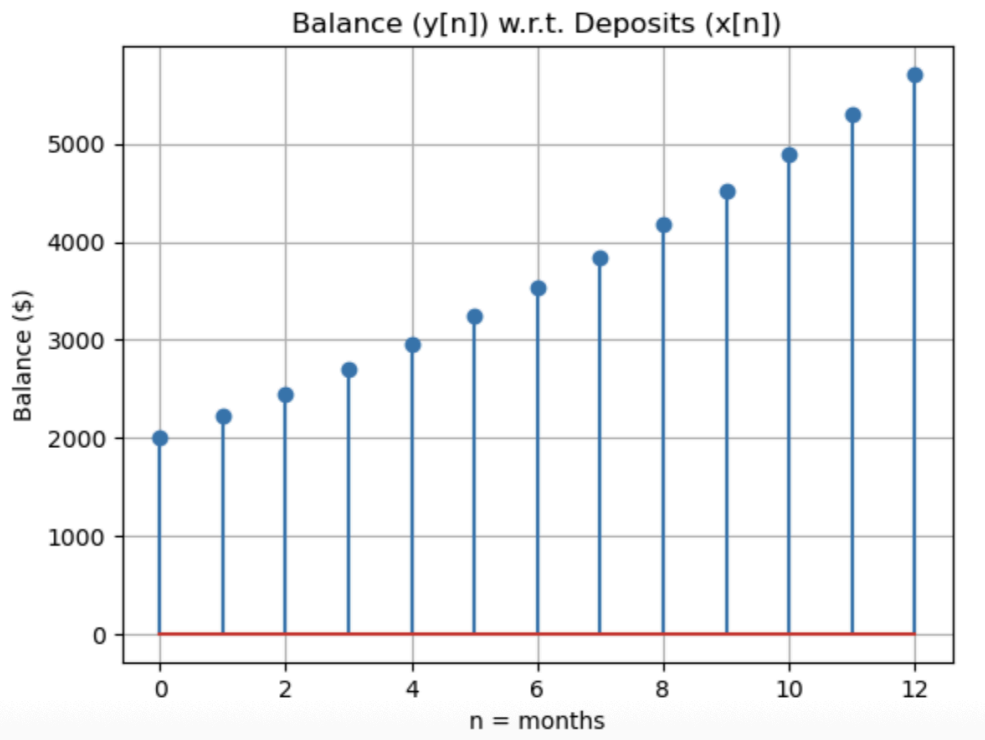


Figure 11: Graphical representation for the equation of $y[n]$

This plot demonstrates an example of \$100 deposited into the bank account each month with an initial deposit of \$2000 and an interest rate of $6\% = 0.06$

Question 2:

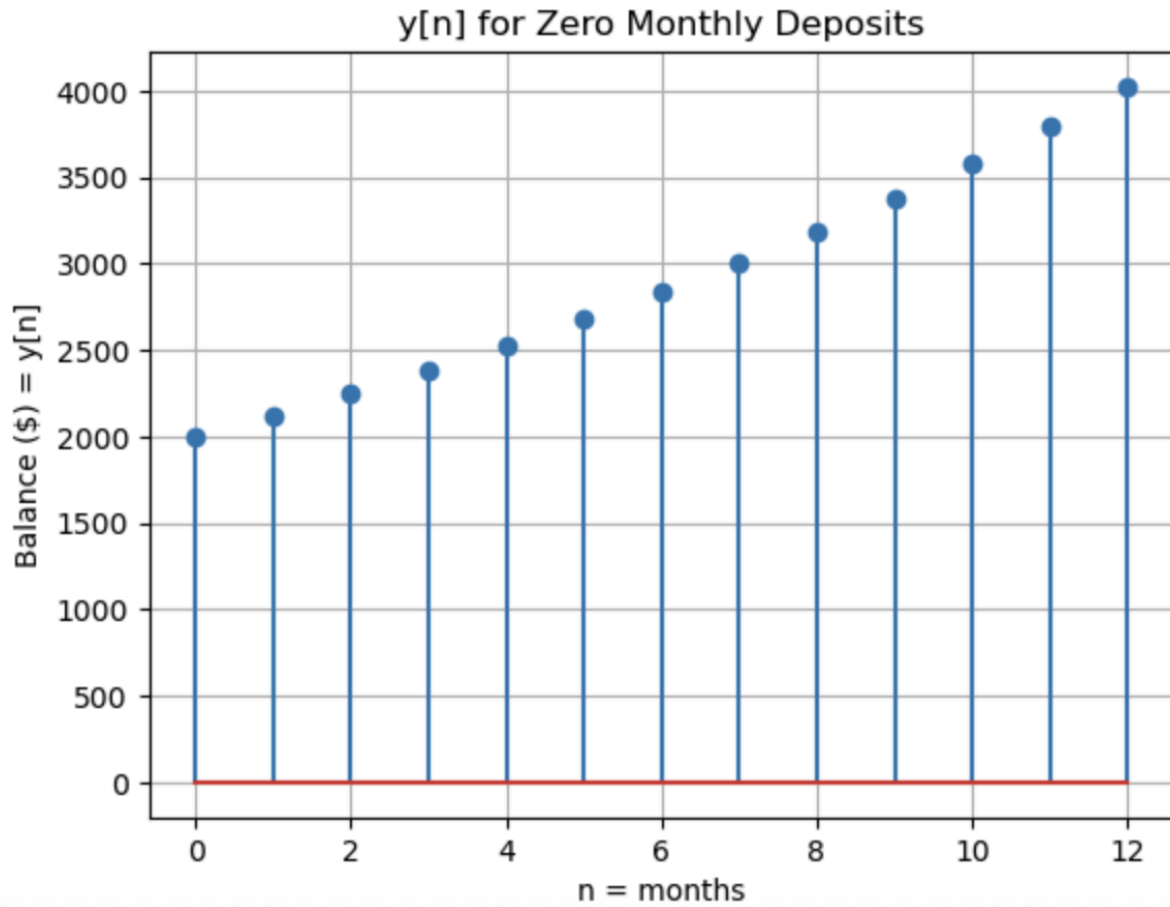


Figure 12: Zero Monthly deposits (Zero-Input Response)

The above plot displays the output, $y[n]$, which is the balance for \$0 deposits made in each month of the year (January - December), although having an account balance of \$2000 for the beginning of the new year. This plot exemplifies the zero-input response for this problem.

Question 3:

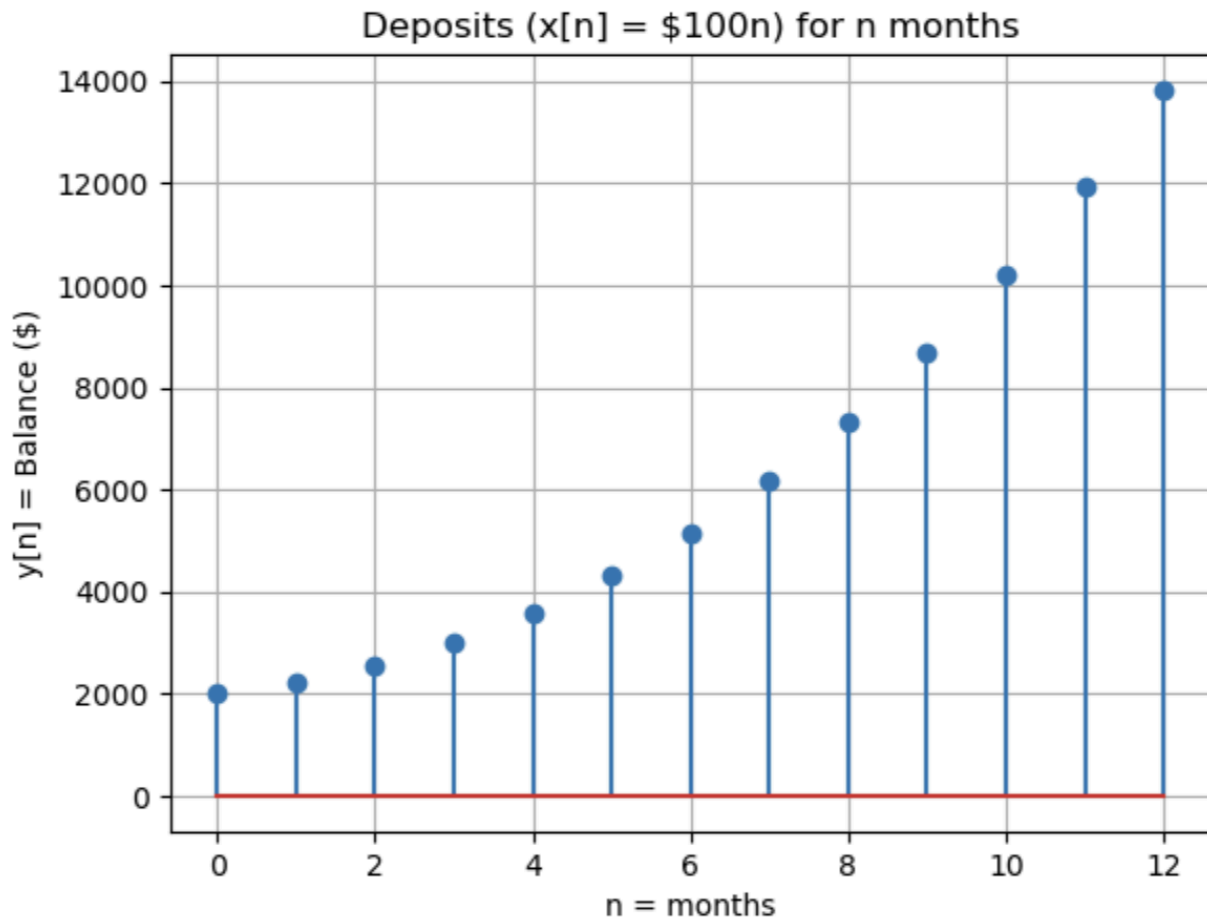


Figure 13: $x[n] = 100n$ (monthly deposits)

The above plot displays the output, $y[n]$, which is the balance for $\$100n$ deposits made in each month of the year ($n=1$ for January to $n=12$ for December), although having an account balance of $\$2000$ for the beginning of the new year. The equation representing the balance correlates to $y[n] = 1.06y[n-1] + x[100n] + 2000$.

PART C: Design a filter (N-point maximum filter)

Question 1:

D = 1 # fourth digit of student ID provided in part B
G = 2 # seventh digit of student ID provided in part B
H = 5 # eighth digit of student ID provided in part B

```
def max_filtering(x, N):  
    y = x  
    for i in range(0, N):  
        y = np.insert(y, 0, 0)  
  
    for j in range(0, 45-N):  
        sample_filter = [-100.0, -100.0]  
        for k in range(0, N):  
            sample_filter = np.insert(sample_filter, len(sample_filter), x[k+j])  
  
        y[j+N] = np.amax(sample_filter)  
  
    for i in range(0, N):  
        y = np.delete(y, len(y)-1)  
  
    return np.array(y)
```

Function 1: Performs maximum filtering on an input vector x with length M

Question 2:

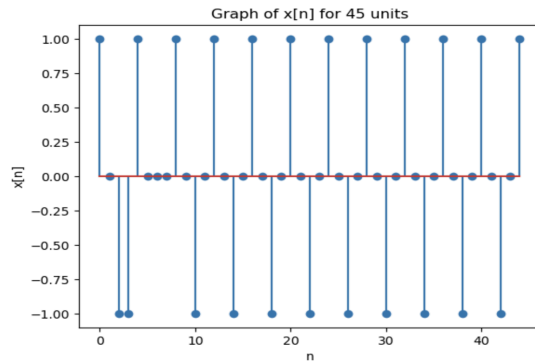


Figure 14: Graph of $x[n]$

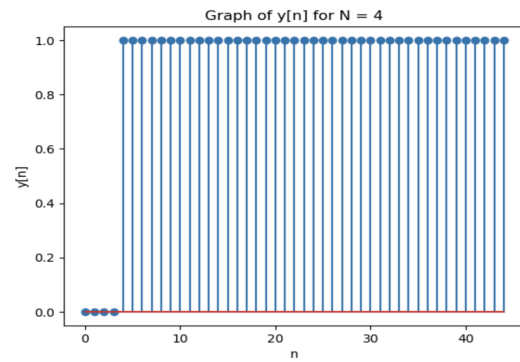


Figure 15: Graph of $y[n]$ for $N=4$

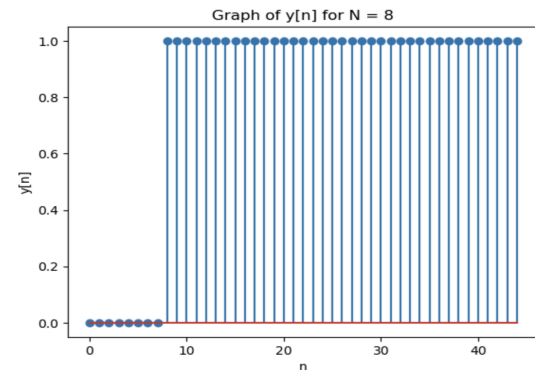


Figure 16: Graph of $y[n]$ for $N=8$

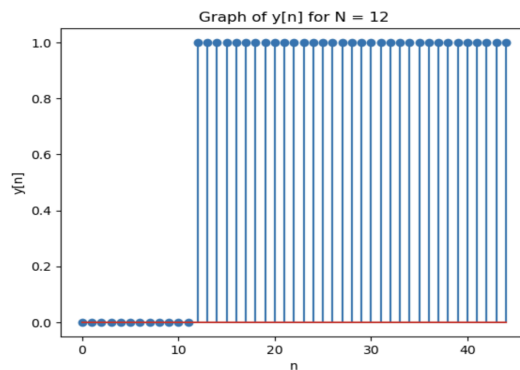


Figure 17: Graph of $y[n]$ for $N=12$

Question 3:

The input signal is given as the following equation:

$$x[n] = \cos\left(\frac{\pi n}{(D-1)}\right) - \delta[n - (G - 1)] + \delta[n - (H + 1)]$$

This is sampled for 45 units (or points), that is, $n = [0, 45)$, where the length of the array, $M=45$. Based on the value of N ($N = 4, 8$, and 12), there are $N-1$ output values of zero (where $x[n] = 0$ for $n = N - 1$). Given the array of outputs for $x[n]$, the maximum value in the array of length, $M=45$ is $x[n] = 1$ since it is a sinusoid. Therefore, the function `np.amax` produces output values for $x[n] = 1$. This is represented for the $N=4$, $N=8$, and $N=12$ plots of $x[n]$, after having the output as zero for the first 3, 7, 11 points, respectively, for $N-1$ values (starting from zero). However, for $x[n]$ the values vary since the signal only takes 4 samples (points of the array) at a time. As noticed in the $x[n]$ plot, the sequence oscillates for every 4 points to the maximum filtered value of 1. Additionally, due to the values assigned in the function based on the student ID where $G=2$ & $H=5$ the corresponding units (n) are valued at zero in the graph of $x[n]$. This is due to the delta function as the units of n are filtered to maximum output or zero values.

PART D: Energy and power of a discrete signal

Question 1:

```
def energy_and_power(x):  
    x = np.asarray(x)  
  
    E = np.sum(np.abs(x)**2)  
    P = E / len(x)  
  
    return E, P
```

Function 2: Receives a finite-length vector $x[n]$ and returns the energy and power of the vector

Question 2:

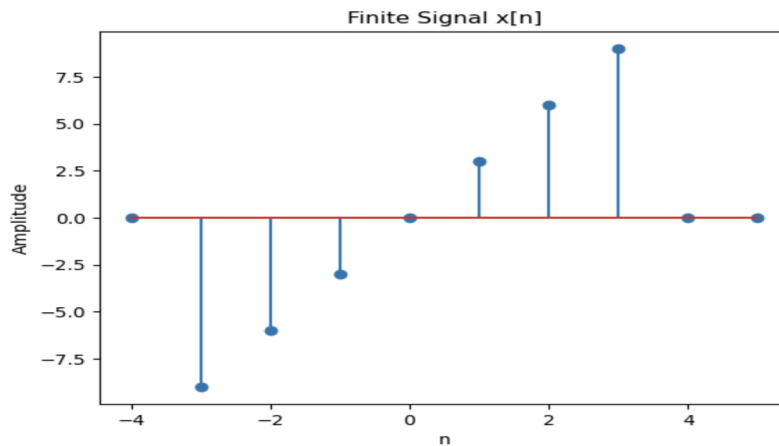


Figure 18: Python implementation of the Finite Signal $x[n]$

A discrete-time signal's total energy is equal to the sum of the squares of each sample's magnitude, as given in equation 2 below.

$$E = \sum_{n=0}^{N-1} |x[n]|^2 \quad [2]$$

The average power is defined as the energy per unit time (or per sample) during the lifetime of the signal, as given in equation 3 below.

$$P = \frac{E}{N} = \frac{\sum_{n=0}^{N-1} |x[n]|^2}{N} \quad [3]$$

```
The Energy (in Joules) of the finite system x[n]: 252.00 J  
The Power (in Watts) of the finite system x[n]: 36.00000000000001 W
```

Figure 19: Python algorithmic calculation of the Finite Signal $x[n]$ energy and power

APPENDIX

PART A

Question 1:

```
n = np.arange(-10, 11)
delta = np.zeros(len(n))
delta[n==3] = 1

plt.stem(n, delta)
plt.xlabel("n")
plt.ylabel(" $\delta[n-3]$ ")
plt.title("Plot of  $\delta[n-3]$ ")
plt.show()
```

```
n = np.arange(-10, 11)
u = np.zeros(len(n))
u[n >= -1] = 1

plt.stem(n, u)
plt.xlabel("n")
plt.ylabel("u[n + 1]")
plt.title("Plot of u[n + 1]")
plt.show()
```

```
n = np.arange(-10, 31)
u = np.zeros(len(n))
u[n >= 0] = 1
x = np.cos(np.pi * n / 5) * u

plt.stem(n, x)
plt.xlabel("n")
plt.ylabel("x[n]")
plt.title("Plot of x[n]")
plt.show()
```

```
n = np.arange(-10, 31)
u = np.zeros(len(n))
u[n >= 0] = 1
x = np.cos(np.pi * n / 5) * u

plt.stem(n+3, x)
plt.xlabel("n")
plt.ylabel("x\u2081[n]")
plt.title("Plot of x\u2081[n]")
plt.show()
```

```

n = np.arange(-31, 31)
u = np.zeros(len(n))
u[n >= 0] = 1
x = np.cos(np.pi * n / 5) * u

plt.stem(-n, x)
plt.xlabel("n")
plt.ylabel("x\u2082[n]")
plt.title("Plot of x\u2082[n]")
plt.show()

```

Question 2:

```

x = np.arange(-10, 71)
u = (x % 1 == 0) * (x >= 0).astype(float)
u2 = (x % 1 == 0) * (x >= 10).astype(float)
y = 5 * np.exp(-x / 8) * (u - u2)

plt.stem(x, y)
plt.xlabel("n")
plt.ylabel("y[n]")
plt.title("Plot of y[n]")
plt.show()

```

```

x = np.arange(-10, 71)
z = 3 * x
u = (z % 1 == 0) * (z >= 0).astype(float)
u2 = (z % 1 == 0) * (z >= 10).astype(float)
y = 5 * np.exp(-3 * x / 8) * (u - u2)

plt.stem(3*x, y)
plt.xlabel("n")
plt.ylabel("y1[n]")
plt.title("Plot of y\u2081[n]")
plt.show()

```

```

x = np.arange(-10, 71)
z = x / 3
u = (z % 1 == 0) * (z >= 0).astype(float)
u2 = (z % 1 == 0) * (z >= 10).astype(float)
y = 5 * np.exp(-x / (8 * 3)) * (u - u2)

plt.stem(x, y)
plt.xlabel("n")
plt.ylabel("y2[n]")
plt.title("Plot of y\u2082[n]")
plt.show()

```

Question 3:

```
x = np.arange(-10,71,0.1)
u = np.where(x >= 0, 1, 0)
u2 = np.where(x >= 10, 1, 0)
z = 5 * np.exp(-x/(8)) * (u-u2)
plt.stem(x/3, z)
plt.title('Sampled signal = z(t)')
plt.show()

y = x / 3
u = np.where(y >= 0, 1, 0)
u2 = np.where(y >= 10, 1, 0)
z = 5 * np.exp(-y/(8)) * (u-u2)
plt.stem(x, z)
plt.title('Transformed signal = y\u2083[n]')
plt.show()
```

PART B

Question 1:

```
r = 0.06 # 0.06 = I + I = (5 + 1) / 100
n = range(0, 13) # 12 months in a year
y = [2000, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0] # 2000 = D+I = I+I * 1000
x = [0, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100] # the deposits, arbitrary value

for k in range(1, len(n)):
    y[k] = (1 + r) * y[k-1] + x[k]

plt.stem(n, y)
plt.title("Balance (y[n]) w.r.t. Deposits (x[n])")
plt.xlabel("n = months")
plt.ylabel("Balance ($)")
plt.grid(True)
plt.show()
```

Question 2:

```
r = 0.06 #  $0.06 = I + I = (5 + 1) / 100$ 
n = range(0, 13) # 12 months in a year
y = [2000, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0] #  $2000 = D + I = I + I * 1000$ 
x = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0] # no deposits

for k in range(1, len(n)):
    y[k] = (1 + r) * y[k-1] + x[k]

plt.stem(n, y)
plt.title("y[n] for Zero Monthly Deposits")
plt.xlabel("n = months")
plt.ylabel("Balance ($) = y[n]")
plt.grid(True)
plt.show()
```

Question 3:

```
r = 0.06 #  $0.06 = I + I = (5 + 1) / 100$ 
n = range(0, 13) # 12 months in a year
y = [2000, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0] #  $2000 = D + I = I + I * 1000$ 
x = [0, 100, 200, 300, 400, 500, 600, 700, 800, 900, 1000, 1100, 1200] #  $100n$  deposits ( $n$  is the month number)

for k in range(1, len(n)):
    y[k] = (1 + r) * y[k-1] + x[k]

plt.stem(n, y)
plt.title('Deposits (x[n] = $100n) for n months')
plt.xlabel('n = months')
plt.ylabel('y[n] = Balance ($)')
plt.grid(True)
plt.show()
```


PART C

Question 2:

```
def delta(n):
    return 1.0*(n==0);

def x_partc(n):
    return np.cos(np.pi*n*(1/(D+1)))-delta(n-(G+1))+delta(n-(H+1))

n = np.linspace(0, 44, 45)
plt.figure()
plt.xlabel('n')
plt.ylabel('x[n]')
plt.title('Graph of x[n] for 45 units')
plt.stem(n, x_partc(n))
plt.show();

plt.figure()
plt.xlabel('n')
plt.ylabel('y[n]')
plt.title('Graph of y[n] for N = 4')
plt.stem(range(0, 45), max_filtering(x_partc(n), 4))

plt.show();
plt.figure()
plt.xlabel('n')
plt.ylabel('y[n]')
plt.title('Graph of y[n] for N = 8')
plt.stem(range(0,45), max_filtering(x_partc(n), 8))

plt.show();
plt.figure()
plt.xlabel('n')
plt.ylabel('y[n]')
plt.title('Graph of y[n] for N = 12')
plt.stem(n, max_filtering(x_partc(n), 12))
plt.show();
```

PART D

Question 2:

```
def energy_power(x):
    x = np.array([-9, -6, -3, 0, 3, 6, 9])
    L = len(x)
    N = L
    E = np.sum(np.abs(x) ** 2)
    P = np.linalg.norm(x) ** 2 / N
    return E, P

x = np.array([0, -9, -6, -3, 0, 3, 6, 9, 0, 0])
n = np.array([-4, -3, -2, -1, 0, 1, 2, 3, 4, 5])

plt.stem(n, x)
plt.title('Finite Signal x[n]')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.show()

n = np.arange(0, 20001)
x = ((n % 1 == 0) * 1.0 * (n >= 10000))
E, P = energy_power(x)

print(f"The Energy (in Joules) of the finite system x[n]: {E:.2f} J")
print(f"The Power (in Watts) of the finite system x[n]: {P} W")
```